

LAB-3

WATTANAPONG SUTTAPAK

Outline

2

- ▶ Solving 8-puzzle game by
 - ▶ Depth-limit Search
 - ▶ Breadth-limit Search
 - ▶ Manhattan Distance
 - ▶ A* Search
- ▶ Assignment

How to search a shortest-step solutions?

3

Start

1	2	3
	4	6
7	5	8

Goal

1	2	3
4	5	6
7	8	

Limit = shortest steps

DFS path = ?

BFS path = ?

1	2	3
4	0	5
6	7	8

1	2	3
4	5	6
7	8	

DFS path = ?

BFS path = ?

Depth-limit Search

4

```
1  function depth_limit_search(initial_state, goal_state):
2      stack = [(initial_state, [])]
3      visited = set()
4      limit = 5
5
6      while stack is not empty:
7          current_state, path = stack.pop()
8
9          if current_state.puzzle is equal to goal_state.puzzle:
10             return path
11
12         visited.add(tuple(current_state.puzzle))
13
14         for direction in ['up', 'down', 'left', 'right']:
15             if length of path == limit:
16                 break
17             new_state = current_state.move(direction)
18
19             if new_state is not null and tuple(new_state.puzzle) not in visited:
20                 stack.append((new_state, path + [direction]))
21
22         print(path)
23
24     return null
```

Breadth-limit Search

5

```
1  function breadth_limit_search(initial_state, goal_state):
2      queue = [(initial_state, [])]
3      visited = set()
4      limit = 5
5
6      while queue is not empty:
7          current_state, path = queue.dequeue()
8
9          if current_state.puzzle is equal to goal_state.puzzle:
10             return path
11
12         visited.add(tuple(current_state.puzzle))
13
14         for direction in ['up', 'down', 'left', 'right']:
15             if length of path == limit:
16                 break
17             new_state = current_state.move(direction)
18
19             if new_state is not null and tuple(new_state.puzzle) not in visited:
20                 queue.append((new_state, path + [direction]))
21
22         print(path)
23
24     return null
```

Greedy Algorithm

Travelling salesman problem

Greedy Algorithm with Nearest Neighbor Search

7

```
2  # Define the cities and distances between them
3  cities = ["Bangkok", "Nakhon Sawan", "Phrae", "Chiang Mai", "Phayao"]
4  distances = {
5      ("Bangkok", "Nakhon Sawan"): 240,
6      ("Bangkok", "Phrae"): 520,
7      ("Bangkok", "Chiang Mai"): 700,
8      ("Bangkok", "Phayao"): 780,
9      ("Nakhon Sawan", "Phrae"): 320,
10     ("Nakhon Sawan", "Chiang Mai"): 460,
11     ("Nakhon Sawan", "Phayao"): 480,
12     ("Phrae", "Chiang Mai"): 200,
13     ("Phrae", "Phayao"): 140,
14     ("Chiang Mai", "Phayao"): 150
15 }

50 # Start at Bangkok
51 start_city = "Bangkok"
52 route, total_distance = greedy_tsp(start_city, cities, distances)
53
54 print(route)
55 print(total_distance)
```

Travelling salesman problem

Greedy Algorithm with Nearest Neighbor Search

8

```
16
17 # Create a function to find the nearest neighbor
18 def find_nearest_neighbor(current_city, unvisited_cities, distances):
19     nearest_city = None
20     min_distance = float('inf')
21     for city in unvisited_cities:
22         distance = distances.get((current_city, city)) \
23             or distances.get((city, current_city))
24         if distance < min_distance:
25             min_distance = distance
26             nearest_city = city
27     return nearest_city, min_distance
```

```
28
29 # Greedy algorithm to solve TSP using the Nearest Neighbor approach
30 def greedy_tsp(start_city, cities, distances):
31     unvisited_cities = set(cities)
32     unvisited_cities.remove(start_city)
33     current_city = start_city
34     route = [start_city]
35     total_distance = 0
36
37     while unvisited_cities:
38         next_city, distance = \
39             find_nearest_neighbor(current_city, unvisited_cities, distances)
40         route.append(next_city)
41         total_distance += distance
42         current_city = next_city
43         unvisited_cities.remove(next_city)
44
45     # Return to the starting city
46     return_distance = distances.get((current_city, start_city)) \
47         or distances.get((start_city, current_city))
48     route.append(start_city)
49     total_distance += return_distance
50
51     return route, total_distance
```


Travelling salesman problem

Greedy Algorithm with GBFS

9

```
68 def heuristic(current_city, unvisited_cities, distances):
69     if not unvisited_cities:
70         return 0
71     nearest_distance = float('inf')
72     for city in unvisited_cities:
73         distance = distances[frozenset([current_city, city])]
74         if distance < nearest_distance:
75             nearest_distance = distance
76     return nearest_distance
```

```
79 def greedy_best_first_search(start_city, cities, distances):
80     unvisited_cities = set(cities)
81     unvisited_cities.remove(start_city)
82     current_city = start_city
83     route = [start_city]
84     total_distance = 0
85
86     while unvisited_cities:
87         # Find the nearest neighbor using the heuristic
88         next_city = None
89         min_heuristic = float('inf')
90         for city in unvisited_cities:
91             distance = distances[frozenset([current_city, city])]
92             h = heuristic(city, unvisited_cities - {city}, distances) + distance
93             if h < min_heuristic:
94                 min_heuristic = h
95                 next_city = city
96
97         # Move to the next city
98         route.append(next_city)
99         total_distance += distances[frozenset([current_city, next_city])]
100        current_city = next_city
101        unvisited_cities.remove(next_city)
102
103        # Return to the starting city
104        return_distance = distances[frozenset([current_city, start_city])]
105        route.append(start_city)
106        total_distance += return_distance
```

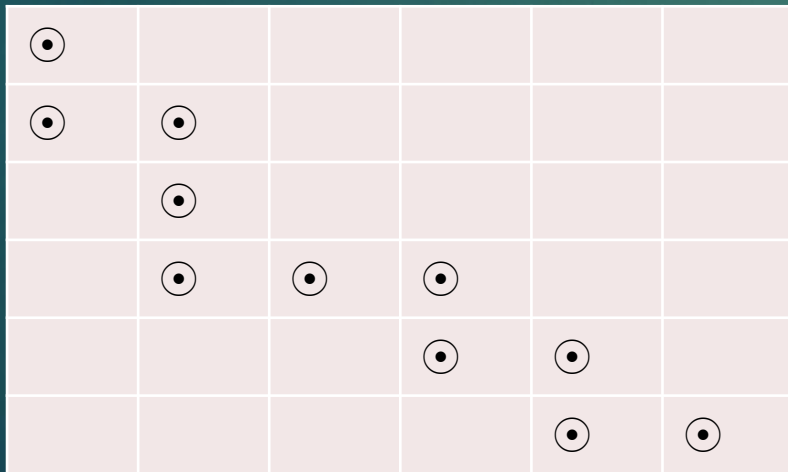
Manhattan Distance

10

- โดยปรกติเรารู้กันเคยกับการคำนวณระยะระหว่างจุดสองจุดด้วย

$$\text{Euclidean distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- ▶ Manhattan or taxicab distance เป็นการหาผลรวมของระยะในแนวแกน x และ y



$$\text{Manhattan distance} = |x| + |y|$$

$$\text{Manhattan distance} = \text{sum of } x + \text{sum of } y$$

$$\text{Manhattan distance} = (1 + 2 + 1 + 1) + (1 + 2 + 1 + 1)$$

$$\text{Manhattan distance} = 5 + 5 = 10$$

A* Search

11

```
1 import numpy as np
2
3 class PuzzleNode:
4     def __init__(self, state, parent=None, action=None, cost=0, heuristic=0):
5         self.state = state
6         self.parent = parent
7         self.action = action
8         self.cost = cost
9         self.heuristic = heuristic
10
11     # less than for instance obj1 < obj2 when obj1 and obj2 are PuzzleNode
12     def __lt__(self, other):
13         return (self.cost + self.heuristic) < (other.cost + other.heuristic)
```

```
15 # manhattan_distance
16 def mh_dist(point1, point2):
17     return abs(point1[0] - point2[0]) + abs(point1[1] - point2[1])
18
19 def find_pos(state, value):
20     for i in range(len(state)):
21         for j in range(len(state[0])):
22             if state[i][j] == value:
23                 return (i, j)
24
25 def get_dist(init, goal):
26     if len(init) != len(goal) or len(init[0]) != len(goal[0]):
27         raise ValueError("States must have the same dimensions")
28
29     manhattan_sum = 0
30     for i in range(len(init)):
31         for j in range(len(init[0])):
32             value = init[i][j]
33             if value != 0: # Skip the empty space
34                 goal_pos = find_pos(goal, value)
35                 manhattan_sum += mh_dist((i, j), goal_pos)
36
37     return manhattan_sum
```

A* Search

12

```
42 def gen_child(node, goal_state):
43     children = []
44     i = 0
45     while 0 not in node.state[i]:
46         i += 1
47
48     row = i
49     col = node.state[i].index(0)
50
51     actions = [(0, 1), (1, 0), (0, -1), (-1, 0)] # Right, Down, Left, Up
52
53     for action in actions:
54         nrow, ncol = (row + action[0], col + action[1])
55
56         if 0 <= nrow < 3 and 0 <= ncol < 3:
57             new_state = [list(row) for row in node.state]
58             new_state[row][col] = new_state[nrow][ncol]
59             new_state[nrow][ncol] = 0
60             child_node = PuzzleNode(
61                 new_state,
62                 parent=node,
63                 action=action,
64                 cost=node.cost + 1,
65                 heuristic=get_dist(new_state, goal_state),
66             )
67             children.append(child_node)
68
69     return children
```

A* Search

13

```
72 def a_star_search(initial_state, goal_state):
73     initial_node = PuzzleNode(
74         initial_state, heuristic=get_dist(initial_state, goal_state)
75     )
76     stack = [initial_node]
77     closed_set = set()
78
79     while stack:
80         current_node = stack.pop()
81
82         if current_node.state == [[1, 2, 3], [4, 5, 6], [7, 8, 0]]:
83             return get_solution_path(current_node)
84
85         closed_set.add(tuple(map(tuple, current_node.state)))
86
87         children = gen_child(current_node, goal_state)
88
89         heuristics = []
90         _children = []
91         for child in children:
92             if tuple(map(tuple, child.state)) not in closed_set:
93                 _children.append(child)
94                 heuristics.append(child.heuristic)
95
96         indx = np.argsort(-1*np.array(heuristics))
97
98         stack.extend([_children[i] for i in indx])
99
100     return None
```

A* Search

14

```
103 def get_solution_path(node):
104     path = []
105     while node.parent is not None:
106         path.insert(0, (node.action, node.state))
107         node = node.parent
108     return path
109
110
111 if __name__ == "__main__":
112     # initial_state = [[2, 3, 5], [1, 4, 6], [7, 8, 0]] # 8 steps
113     # initial_state = [[1, 3, 2], [4, 5, 6], [7, 8, 0]] # no solution
114     initial_state = [[1, 2, 3], [4, 0, 5], [6, 7, 8]] # 818 steps
115     # initial_state = [[1, 2, 3], [0, 4, 6], [7, 5, 8]] # 8 steps
116     goal_state = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]
117     solution_path = a_star_search(initial_state, goal_state)
118
119     if solution_path:
120
121         for step in solution_path:
122             print(f"Move {step[0]}:")
123             for row in step[1]:
124                 print(row)
125             print("-----")
126
127         print(f"Solution found in {len(solution_path)} moves")
128     else:
129         print("No solution found.")
```